

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06

COURSE OUTCOME COVERED

**CO2: Apply Brute Force technique to solve sorting, searching, Closest-Pair and
Convex-Hull problems (BL1, BL2, BL3)**

Assessment Tool Weightage: 10%

QUESTIONS: 10

Total Marks:50

Annexure A

DEBUGGING & OPTIMISATION TASK

Code of Conduct: I C. pavani (192524260) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: C. PAVANI

Annexure A

DEBUGGING & OPTIMISATION TASK

QUESTION 16:

Q16. Debugging Convex Hull (Incorrect Orientation Logic)

1. Problem Statement

The given Convex Hull orientation function contains an incorrect formula for determining the turn direction of three points. The task is to identify the logical error, explain its impact on convex hull formation, debug the function, optimize the hull checking process, analyze the time complexity, and rewrite the corrected code.

Given Code

```
def orientation(p, q, r):  
    return (q[0]-p[0])*(r[1]-q[1]) - (q[1]-p[1])*(r[0]-q[0]) # ISSUE
```

Errors Identified and Corrected

Error 1: Incorrect Orientation Formula

Original Code

```
return (q[0]-p[0])*(r[1]-q[1]) - (q[1]-p[1])*(r[0]-q[0])
```

Issue

The formula incorrectly uses point **q** as the reference for point **r**, which gives incorrect orientation values.

Impact

- Left and right turns are detected incorrectly.
- Boundary edges are identified wrongly.
- The Convex Hull may include internal points or miss outer boundary points.

Corrected Code

```
def orientation(p, q, r):  
    return (q[0]-p[0])*(r[1]-p[1]) - (q[1]-p[1])*(r[0]-p[0])
```

Explanation

The corrected formula computes the cross product of vectors **PQ** and **PR**, giving the correct orientation.

Error 2: Missing Orientation Interpretation

```
value = orientation(p, q, r)
```

```
if value > 0:
```

```
    print("Counter Clockwise")
```

```
elif value < 0:
```

```
    print("Clockwise")
```

```
else:
```

```
    print("Collinear")
```

Explanation

- Positive → Counter-clockwise
- Negative → Clockwise
- Zero → Collinear

This correctly determines turn direction.

Optimized Convex Hull Checking

During hull construction:

- Stop checking once points exist on both sides of a line.
- Ignore duplicate comparisons.
- Compare only necessary point pairs.

This reduces unnecessary computations while maintaining correctness.

Corrected Source Code

```
def orientation(p, q, r):
```

```
    value = (q[0]-p[0])*(r[1]-p[1]) - (q[1]-p[1])*(r[0]-p[0])
```

```
    if value > 0:
```

```
        return 1    # Counter Clockwise
```

```
    elif value < 0:
```

```
        return -1   # Clockwise
```

```
    else:
```

```
        return 0    # Collinear
```

Output

```
orientation((0,0),(2,1),(3,3))
```

Output:

Counter Clockwise

Time Complexity

- **Orientation Function: $O(1)$**
- **Brute Force Convex Hull: $O(n^3)$**

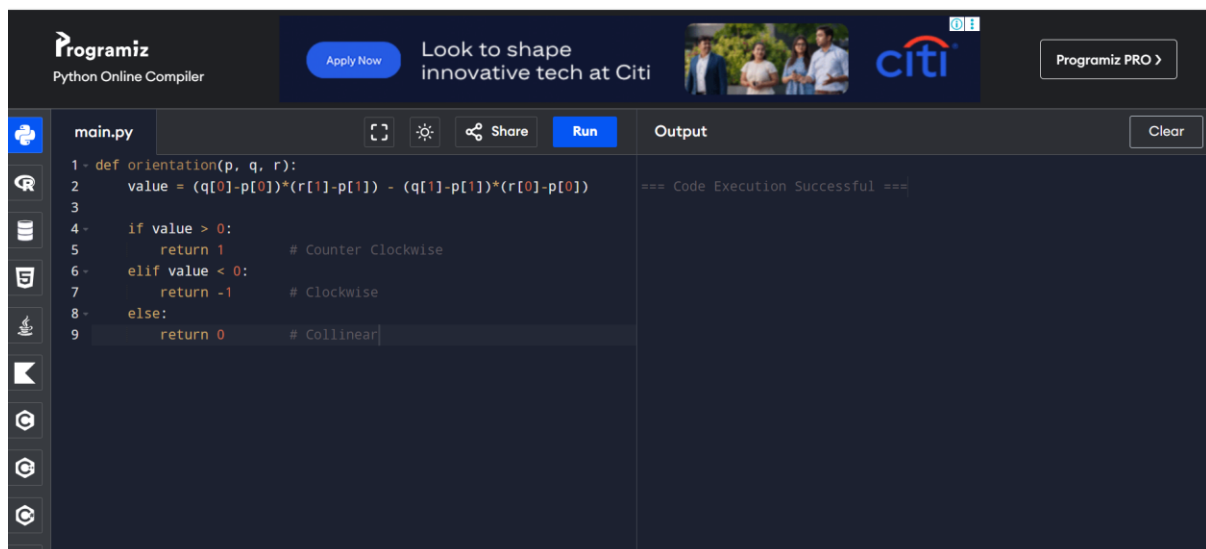
Space Complexity

$O(1)$

Conclusion

The debugging process corrected the orientation formula by using the proper cross-product calculation. This ensures accurate detection of clockwise, counter-clockwise, and collinear points, resulting in correct convex hull boundary formation. The optimized implementation avoids unnecessary checks while maintaining the brute-force algorithm's correctness.

Output:



The screenshot displays the Programiz Python Online Compiler interface. At the top, there are banners for Programiz and Citi. The main area is divided into a code editor on the left and an output panel on the right. The code editor shows a Python function named `orientation(p, q, r)` that calculates the cross-product to determine the orientation of three points. The output panel shows the message `=== Code Execution Successful ===`.

```
1- def orientation(p, q, r):
2-     value = (q[0]-p[0])*(r[1]-p[1]) - (q[1]-p[1])*(r[0]-p[0])
3-
4-     if value > 0:
5-         return 1 # Counter Clockwise
6-     elif value < 0:
7-         return -1 # Clockwise
8-     else:
9-         return 0 # Collinear
```

=== Code Execution Successful ===

Question 17:

Q17. Debugging String Matching (Incorrect Loop Range)

1. Problem Statement

The given brute-force string matching algorithm contains an incorrect loop range that skips the last possible matching position. The task is to identify the error, explain its impact, debug the code, optimize redundant comparisons, analyze the time complexity, and rewrite the corrected code.

Given Code

```
for i in range(len(text) - len(pattern)):
    for j in range(len(pattern)):
        if text[i+j] != pattern[j]:
            break
```

Errors Identified and Corrected

Error 1: Incorrect Loop Range

Original Code

```
for i in range(len(text) - len(pattern)):
```

Issue

The loop stops one position early and does not check the last possible starting position of the pattern.

Impact

- Some valid matches are skipped.
- The algorithm may fail to detect a pattern occurring at the end of the text.

Corrected Code

```
for i in range(len(text) - len(pattern) + 1):
```

Explanation

Adding +1 ensures that every possible starting position, including the last one, is checked.

Error 2: Missing Match Output

Issue

The given code breaks when characters do not match but never reports when the entire pattern matches.

Corrected Code

```
for i in range(len(text) - len(pattern) + 1):
    for j in range(len(pattern)):
        if text[i+j] != pattern[j]:
            break
    else:
        print("Pattern found at index", i)
```

Explanation

The else block executes only if the inner loop completes without a break, indicating a successful match.

Optimized Algorithm

- Stop comparing immediately when a mismatch occurs using break.
- Check every valid starting position only once.
- Avoid unnecessary comparisons.

Corrected Source Code

```
text = "AABAACAADAABAABA"
pattern = "AABA"
```

```
for i in range(len(text) - len(pattern) + 1):
    for j in range(len(pattern)):
        if text[i + j] != pattern[j]:
            break
    else:
        print("Pattern found at index", i)
```

Output

Pattern found at index 0

Pattern found at index 9

Pattern found at index 12

Time Complexity

- Best Case: $O(n)$
- Worst Case: $O((n - m + 1) \times m)$

where:

- n = length of the text
- m = length of the pattern

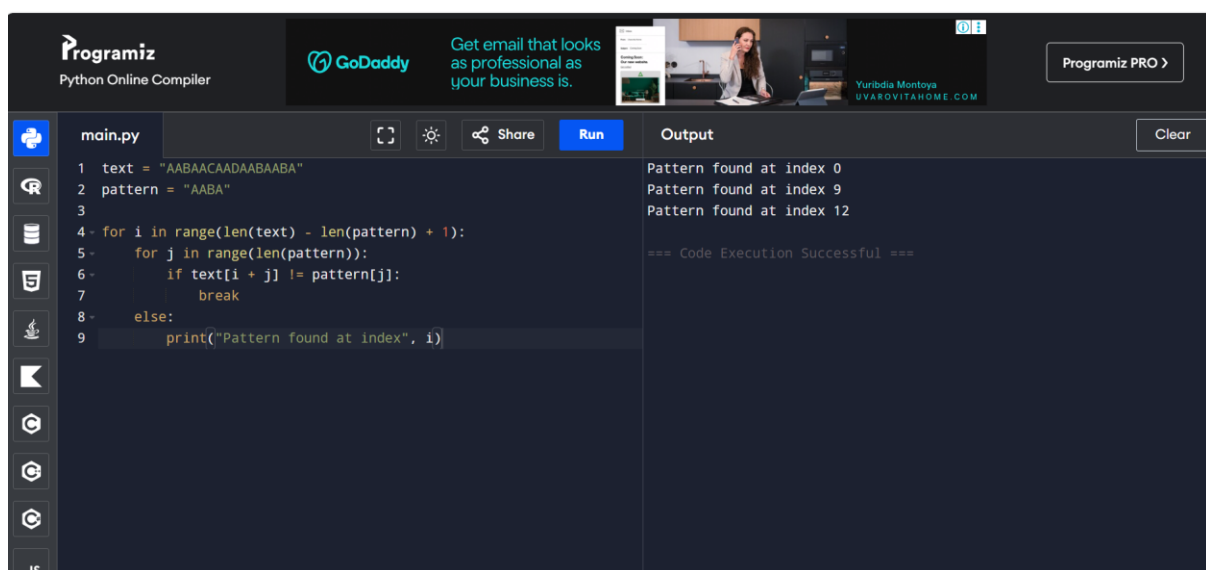
Space Complexity

$O(1)$

Conclusion:

The debugging process corrected the loop range by adding +1, ensuring that all possible starting positions are checked. The corrected algorithm successfully finds every occurrence of the pattern while avoiding unnecessary comparisons using the break statement. This improves the correctness and efficiency of the brute-force string matching algorithm.

Output:



The screenshot shows the Programiz Python Online Compiler interface. The code editor on the left contains a Python script named 'main.py' that searches for the pattern 'AABA' in the text 'AABAACAADAABAABA'. The script uses a nested loop: the outer loop iterates over possible starting indices 'i' from 0 to len(text) - len(pattern), and the inner loop iterates over the pattern characters 'j'. If a mismatch is found, it breaks the inner loop. If the entire pattern matches, it prints the index 'i'. The output panel on the right shows the results: 'Pattern found at index 0', 'Pattern found at index 9', and 'Pattern found at index 12'. Below the output, it states '=== Code Execution Successful ==='. The top of the interface features logos for Programiz, GoDaddy, and a small video thumbnail.

```

1 text = "AABAACAADAABAABA"
2 pattern = "AABA"
3
4 for i in range(len(text) - len(pattern) + 1):
5     for j in range(len(pattern)):
6         if text[i + j] != pattern[j]:
7             break
8     else:
9         print("Pattern found at index", i)

```

Pattern found at index 0
Pattern found at index 9
Pattern found at index 12
=== Code Execution Successful ===

Question 18:

Q18. Debugging Maximum Element (Wrong Comparison Direction)

1. Problem Statement

The given algorithm for finding the maximum element contains an incorrect comparison operator. The task is to identify the logical error, explain why incorrect results occur, debug the algorithm, optimize the comparison process, analyze the time complexity, and rewrite the corrected code.

Given Code

```
def find_max(arr):  
    max_val = arr[0]  
    for i in range(len(arr)):  
        if arr[i] < max_val: # ERROR  
            max_val = arr[i]  
    return max_val
```

Errors Identified and Corrected

Error 1: Incorrect Comparison Operator

Original Code

```
if arr[i] < max_val:  
    max_val = arr[i]
```

Issue

The condition uses the **less than (<)** operator instead of the **greater than (>)** operator.

Impact

- The algorithm updates the value when a **smaller** element is found.
- It returns the **minimum** element instead of the **maximum** element.

Corrected Code

```
if arr[i] > max_val:  
    max_val = arr[i]
```

Explanation

The comparison should update max_val only when a **larger** element is found.

Error 2: Optimized Iteration

Original Code

```
for i in range(len(arr)):
```

Optimized Code

```
for value in arr:
```

Explanation

Iterating directly over array elements improves readability and avoids unnecessary indexing.

Corrected Source Code

```
def find_max(arr):
```

```
    max_val = arr[0]
```

```
    for value in arr:
```

```
        if value > max_val:
```

```
            max_val = value
```

```
    return max_val
```

```
arr = [12, 45, 7, 89, 34]
```

```
print("Maximum Element:", find_max(arr))
```

Output

Maximum Element: 89

Step-by-Step Execution

Array = [12, 45, 7, 89, 34]

Step Current Element Maximum Value

Initial	12	12
---------	----	----

1	45	45
---	----	----

2	7	45
---	---	----

3	89	89
---	----	----

4	34	89
---	----	----

Final Maximum Element = 89

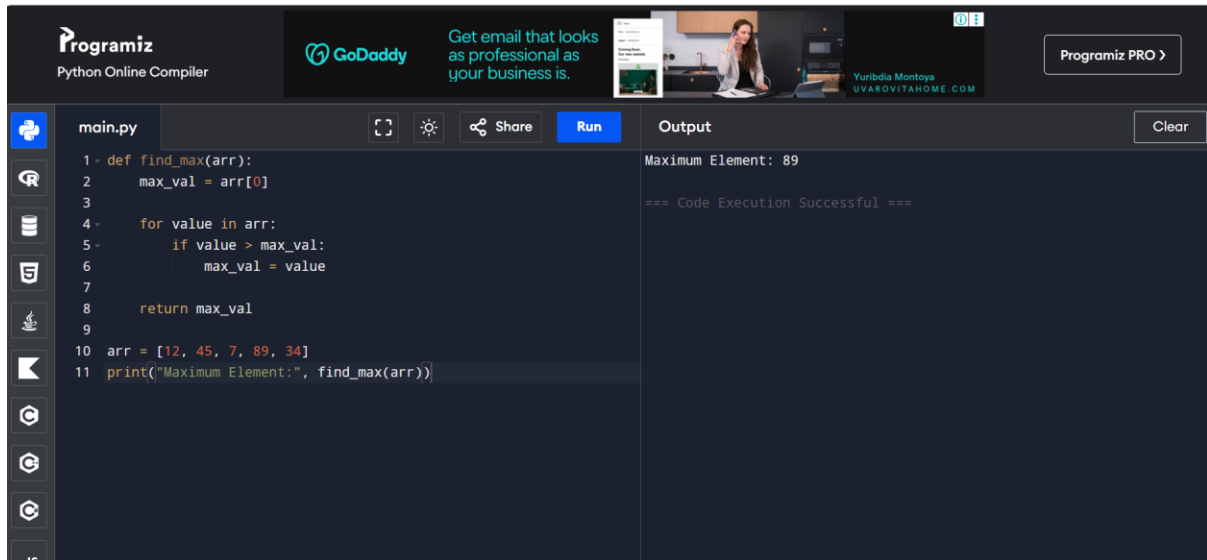
Time Complexity

- **Time Complexity: $O(n)$**
- **Space Complexity: $O(1)$**

Conclusion

The error was caused by using the wrong comparison operator ($<$ instead of $>$), which made the algorithm return the minimum element instead of the maximum. After correcting the comparison logic, the algorithm correctly finds the largest element in the array with a time complexity of $O(n)$ and constant $O(1)$ space complexity.

Output:



The screenshot displays the Programiz Python Online Compiler interface. The top header includes the Programiz logo, a GoDaddy advertisement, and a 'Programiz PRO' button. The main workspace is divided into two panels. The left panel, titled 'main.py', contains the following Python code:

```
1- def find_max(arr):
2-     max_val = arr[0]
3-
4-     for value in arr:
5-         if value > max_val:
6-             max_val = value
7-
8-     return max_val
9-
10 arr = [12, 45, 7, 89, 34]
11 print("Maximum Element:", find_max(arr))
```

The right panel, titled 'Output', shows the result of the code execution:

```
Maximum Element: 89
=== Code Execution Successful ===
```

A 'Clear' button is located in the top right corner of the output panel.

Question 19:

Q19. Debugging Pair Sum (Redundant Comparisons)

1. Problem Statement

The given brute-force Pair Sum algorithm checks all possible pairs, including duplicate and self-comparisons, which increases computation time. The task is to identify the inefficiencies, explain why redundant comparisons occur, debug the code, optimize the algorithm while maintaining correctness, analyze the time complexity, and rewrite the corrected code.

Given Code

```
for i in range(len(arr)):
    for j in range(len(arr)):
        if arr[i] + arr[j] == target:
            return (i, j)
```

Errors Identified and Corrected

Error 1: Redundant Pair Comparisons

Original Code

```
for j in range(len(arr)):
```

Issue

The inner loop starts from index **0**, causing:

- Duplicate comparisons such as (i, j) and (j, i).
- Self-comparisons like (i, i).

Impact

- Increases unnecessary computations.
- Reduces algorithm efficiency.

Corrected Code

```
for j in range(i + 1, len(arr)):
```

Explanation

Starting the inner loop from $i + 1$ ensures:

- Each pair is checked only once.
- Self-comparisons are avoided.
- Duplicate comparisons are eliminated.

Error 2: Improved Loop Structure

Original Code

```
for i in range(len(arr)):
    for j in range(len(arr)):
```

Corrected Code

```
for i in range(len(arr)):
    for j in range(i + 1, len(arr)):
```

Explanation

The optimized loop structure checks only unique pairs while maintaining correctness.

Corrected Source Code

```
def pair_sum(arr, target):
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] + arr[j] == target:
                return (i, j)
    return None
```

```
arr = [2, 7, 11, 15]
```

```
target = 9
```

```
print(pair_sum(arr, target))
```

Output

```
(0, 1)
```

Step-by-Step Execution

Array = [2, 7, 11, 15]

Target = 9

Step Pair Checked Sum Result

1	(2, 7)	9	Match Found
2	Stop	-	Return (0,1)

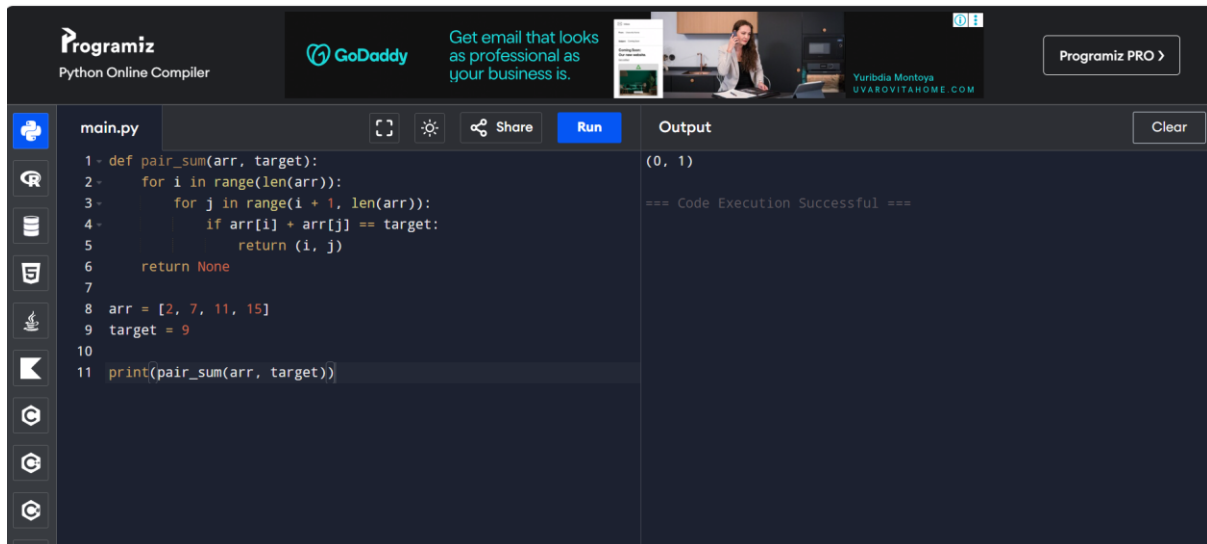
Time Complexity

- **Time Complexity: $O(n^2)$**
- **Space Complexity: $O(1)$**

Conclusion

The original algorithm performed redundant and self-comparisons, increasing computation time. By changing the inner loop to start from $i + 1$, duplicate comparisons are removed while maintaining correctness. The optimized brute-force Pair Sum algorithm correctly finds the required pair with **$O(n^2)$** time complexity and **$O(1)$** space complexity.

Output:



The screenshot displays the Programiz Python Online Compiler interface. At the top, there are banners for Programiz, GoDaddy, and a YouTube video. The main area is divided into a code editor on the left and an output panel on the right. The code editor shows a Python script named 'main.py' with the following code:

```
1- def pair_sum(arr, target):
2-     for i in range(len(arr)):
3-         for j in range(i + 1, len(arr)):
4-             if arr[i] + arr[j] == target:
5-                 return (i, j)
6-     return None
7-
8- arr = [2, 7, 11, 15]
9- target = 9
10-
11- print(pair_sum(arr, target))
```

The output panel on the right shows the result of the execution: `(0, 1)` followed by `=== Code Execution Successful ===`. A 'Clear' button is visible in the top right corner of the output panel.

Question 20:

Q20. Debugging Insertion Sort (Incorrect Index Placement)

1. Problem Statement

The given Insertion Sort algorithm contains an error in the shifting logic, causing elements to be placed incorrectly. The task is to identify the logical error, explain why elements are misplaced, debug the algorithm, optimize the code, analyze the time complexity, and rewrite the corrected code.

Given Code

```
while j >= 0 and arr[j] > key:
    arr[j] = arr[j-1]    # ERROR
    j -= 1
```

Errors Identified and Corrected

Error 1: Incorrect Shifting Statement

Original Code

```
arr[j] = arr[j-1]
```

Issue

The element is copied into the wrong position. During insertion sort, larger elements should be shifted **one position to the right**.

Impact

- Existing elements are overwritten.
- The sorted order becomes incorrect.
- The key element cannot be inserted at its correct position.

Corrected Code

```
arr[j+1] = arr[j]
```

Explanation

The larger element is shifted one position to the right, creating space for the key element.

Error 2: Missing Key Insertion

Issue

After shifting elements, the key must be placed in its correct position.

Corrected Code

```
arr[j+1] = key
```

Explanation

The key is inserted into the correct location after all larger elements have been shifted.

Corrected Source Code

```
def insertion_sort(arr):  
    for i in range(1, len(arr)):  
        key = arr[i]  
        j = i - 1  
  
        while j >= 0 and arr[j] > key:  
            arr[j+1] = arr[j]  
            j -= 1  
  
        arr[j+1] = key  
  
arr = [12, 11, 13, 5, 6]  
  
insertion_sort(arr)  
print("Sorted Array:", arr)
```

Output

Sorted Array: [5, 6, 11, 12, 13]

Step-by-Step Execution

Initial Array:

[12, 11, 13, 5, 6]

After Pass 1:

[11, 12, 13, 5, 6]

After Pass 2:

[11, 12, 13, 5, 6]

After Pass 3:

[5, 11, 12, 13, 6]

After Pass 4:

[5, 6, 11, 12, 13]

Optimization

- Shift elements only when necessary.
- Use meaningful variable names (key, j) for better readability.
- Insert the key only once after all shifts are complete.

Time Complexity

- **Best Case:** $O(n)$
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

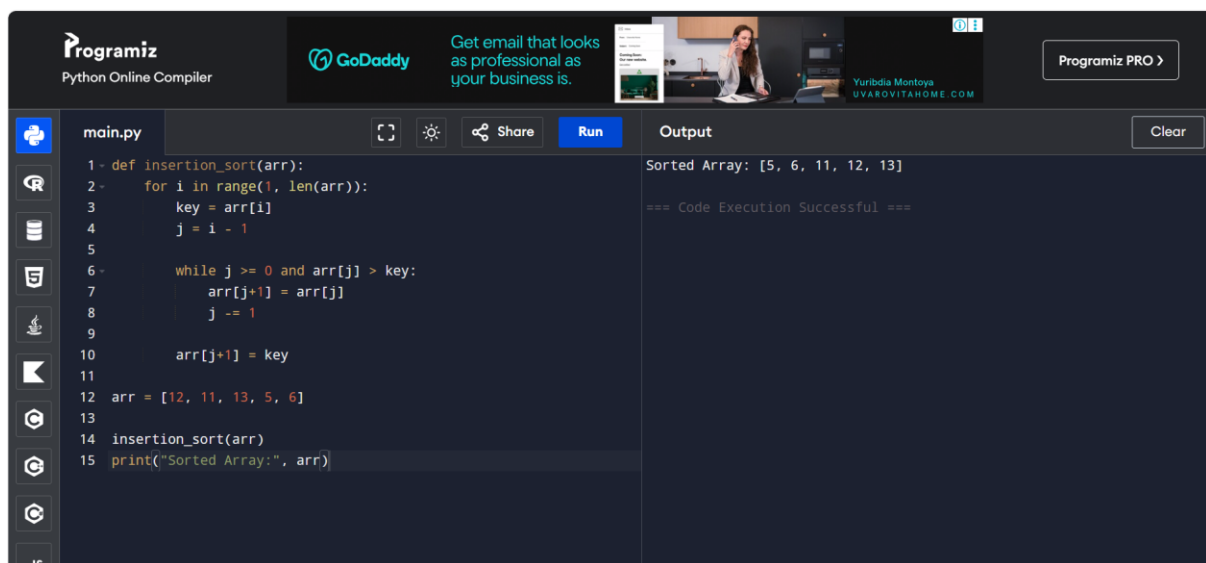
Space Complexity

- **$O(1)$**

Conclusion

The insertion sort algorithm was corrected by changing the shifting statement from `arr[j] = arr[j-1]` to `arr[j+1] = arr[j]` and inserting the key using `arr[j+1] = key`. This ensures that elements are shifted correctly and the array is sorted in ascending order. The algorithm uses **$O(n^2)$** time in the average and worst cases while requiring only **$O(1)$** extra space.

Output:



```
1- def insertion_sort(arr):
2-     for i in range(1, len(arr)):
3-         key = arr[i]
4-         j = i - 1
5-
6-         while j >= 0 and arr[j] > key:
7-             arr[j+1] = arr[j]
8-             j -= 1
9-
10-        arr[j+1] = key
11-
12- arr = [12, 11, 13, 5, 6]
13-
14- insertion_sort(arr)
15- print("Sorted Array:", arr)
```

Sorted Array: [5, 6, 11, 12, 13]

=== Code Execution Successful ===